Decipher

A Foundational Architecture for Human Experience Computation

Authored by Raheem Asghar, Founder - Decipher Prepared for technical and strategic leadership

---

A Note From the Author

This document represents two years of foundational work pursuing a single question: why do organisations keep re-discovering the same problems with human experience, year after year, without ever structurally resolving them?

What began as a frustration with how CX systems were built evolved into an architectural thesis — and then into a working computational model. What follows is my attempt to explain what I have built, why I built it, and why I believe the timing has never been more important.

— Raheem Asghar

The Problem That Has No Name

For over two decades, organisations have known that certain operational surfaces determine whether a human experience succeeds or fails. They called them *drivers*. Drivers of satisfaction. Drivers of loyalty. Drivers of churn.

The intuition was always correct.

But drivers were never made real inside systems. They existed as analytical conclusions — thematic labels that drifted across quarters, reset with every reporting cycle, and vanished the moment the deck was filed. The same surfaces were re-identified, re-debated, and re-reported. Year after year.

This is not a failure of effort or intelligence. It is a structural failure.

Current experience systems were built to *observe*. They were never built to *persist, govern, or learn*.

The result is that organisations have become extraordinarily capable at describing their problems — and structurally incapable of accumulating knowledge about them.

Decipher addresses this at the architectural level.

The Core Claim

> Experience has been observed. It has never been inhabited by machines.

A complaint about triage delays is not the phenomenon. The phenomenon is the condition of *Emergency Services → Triage Efficiency* as a real operational surface in the world. The complaint is evidence that updates our knowledge of the driver's state — but the driver exists independently, persists across time, and can improve or deteriorate regardless of whether anyone is currently reporting on it.

Current systems treat the signal as the unit of reality.

Decipher treats the *driver* as the unit of reality, and the signal as evidence that updates the driver's state.

That is a different ontology. And it changes everything that follows.

The Primitive

Every durable computational system begins with a carefully chosen primitive — the smallest meaningful unit upon which all higher-order logic is built.

Relational databases chose the *row*. Git chose the *commit*. Bitcoin chose the *transaction output*.

Decipher's primitive is the **Experience Driver**.

Formally expressed as `Category → Subcategory`:

- `Emergency Services → Triage Efficiency`
- `Billing → Invoice Clarity`
- `Prescription Management → Refill Authorization`
- `Test Results → Communication Timeliness`

An Experience Driver is not a theme, a score, a sentiment label, or a dashboard category. It is the underlying operational surface where experience actually occurs — the causal determinant that teams own, can act on, and are accountable for.

The Experience Driver was always the right intuition in CX. What was missing was the computational substrate to make it real.

Decipher provides that substrate.

The Three Structural Laws

To make Experience Drivers computationally real, three invariants must be enforced at the schema level. These are not guidelines. They are the laws that make the system a ledger rather than a log.

Identity Law Each Experience Driver has exactly one canonical state container per analysis window. No duplicates. No parallel interpretations. No competing truths. One driver, one state.

Lineage Law Every intervention is a timestamped state transition linked to the driver state that triggered it. Every action traces back to its cause. Every outcome traces forward to its consequence. Nothing happens without being recorded.

Memory Law Every state transition retains referential integrity across time. No driver state is ever orphaned. Historical outcomes become priors for future decisions. The system does not reset.

These three laws are the Experience Driver analogue of double-entry bookkeeping. They are what transform a collection of records into an institutional ledger.

The Architecture

Decipher operates as a layered execution substrate. Each layer has a single responsibility and produces a single class of output.

Layer 1 — Signal Refinement Raw human signals — surveys, support transcripts, reviews, clinical feedback — are processed through a governed taxonomy. One raw comment typically produces multiple fully-structured Experience Driver mentions, each with its own emotional axis, intent classification, journey stage, interaction moment, and behavioral context. Context is preserved, not collapsed.

One narrative about a hospital visit may contain four distinct operational realities, each mapped to a different driver, each requiring a different response.

Layer 2 — State Computation All mentions for a given Experience Driver within a defined window are aggregated into a single **Structured Experience Unit (SEU)**. The SEU is the authoritative state container for that driver. It computes:

- ERI (Emotional Resonance Index) — weighted polarity
- EVI (Emotional Volatility Index) — emotional dispersion
- RF (Recency × Frequency) — urgency composite
- Emotional State Band (ES1 Secure Loyalty through ES5 Critical Failure)
- Temporal intelligence — trajectory, not just snapshot

The SEU answers one question deterministically: *what is the current condition of this driver?* The same inputs always produce identical state. This is not inference. It is computation.

Layer 2.5 — Mission Generation Orchestration Units (OUs) are generated by clustering mentions — not from the aggregated SEU. This is a critical architectural decision. Aggregation destroys behavioral specificity. A single driver may be failing in three distinct ways simultaneously. Each distinct failure pattern produces a separate OU — an atomic, evidence-backed, behaviorally specific mission unit with its own interaction moment, matters statement, and action classification (Fix, Optimize, Amplify, Innovate).

One driver → one SEU → many OUs. State and mission are separated by design.

Layer 3 — Execution Each OU is converted into a locked Problem Statement — a structured definition of what needs to be addressed, why, and what the consequence of inaction is. After this point, reinterpretation is closed.

The Problem Statement initiates a PDCA cycle. In production, this cycle is not powered by generic intelligence — it is powered by the enterprise's own knowledge bases, accessed by an agent under organisational governance. Clinical protocols, SOPs, regulatory requirements, ownership structures, budget constraints, and past intervention outcomes all feed the plan. The result is not an institutionally plausible recommendation. It is an institutionally *true* one.

The Measurement Layer After each intervention, the SEU is recomputed. Before and after states are compared. Elasticity — the proportional state shift relative to effort — is calculated and stored. Impact is derived computationally, not self-reported. This is what makes the system's memory meaningful rather than decorative.

The Institutional Memory Engine (IME) All state transitions, interventions, and outcome evaluations are persisted with full referential integrity. The IME accumulates which interventions worked, under which SEU conditions, for which

driver types, at which urgency levels. Over time, the system doesn't just execute — it develops genuine priors. Past successes constrain future decisions. Patterns that humans forget, the system remembers.

The Emotional Relational Model

The ERM is the relational enforcement layer that makes all of the above governable at scale.

It is not application logic. It is the schema-level substrate that enforces the three laws across the entire architecture. Foreign keys enforce lineage. Uniqueness constraints enforce identity. Append-only event tables enforce memory.

The ERM is to Decipher what the relational model was to data management — not a product, but the formal structure that makes the domain computable.

Its properties emerge directly from the laws:

- Full auditability — every action traces to its trigger and outcome
- Agent-native state — agents query shared deterministic truth, not re-inferred interpretation
- Elasticity verification — provable causality between intervention and state change
- Model independence — swap any LLM without rewriting state
- Horizontal scalability — standard relational operations at any volume
- Zero lock-in — portable SQL schema, the organisation owns its ledger

---

Why Now

Three enabling conditions have converged that make this architecture both buildable and necessary at this precise moment.

LLMs made Layer 1 possible. The extraction of fully-dimensional Experience Driver mentions from unstructured feedback at scale requires semantic understanding that only became reliable with large language models. The architecture uses LLMs precisely where language understanding is required — and locks the output into deterministic relational structure before anything consequential happens. LLMs enable the architecture. They are not the architecture.

The agentic wave created the demand. Before autonomous systems, humans mediated all experience action. Vague recommendations were acceptable

because a human would add context and judgment. Agents cannot hallucinate driver state and act safely. They need stable canonical entities, deterministic shared truth, atomic executable missions, and persistent memory. Decipher provides exactly this environment. It is not merely compatible with agentic AI — it is the structured substrate that agentic AI requires to operate safely and cumulatively on human experience.

Infrastructure maturity made it economical. The ERM requires persistent state across potentially billions of mentions, complex temporal queries, and full referential integrity. Modern cloud infrastructure makes this tractable at a cost that was prohibitive a decade ago.

These three conditions did not exist simultaneously until now. The architecture could not have been built before. It must be built now — because the agentic systems that need this substrate are already being deployed into environments that cannot support them properly.

The Bayesian Organisation

The deepest way to understand what Decipher is building toward is through the lens of organisational learning.

A Bayesian system improves by forming a belief, acting, observing the result, and updating the belief. Most organisations approximate the first step and occasionally attempt the second. They almost never observe the result in a structured way and effectively never update a persistent belief — because the belief has no persistent form. Each cycle begins from scratch.

Decipher makes organisational Bayesian updating structurally possible for the first time in the experience domain.

The SEU is the belief. The OU and PDCA cycle are the action. The outcome evaluation is the observation. The IME is the update. The three laws ensure the belief persists, the action is traceable, the observation is causally linked, and the update compounds.

When this loop closes reliably, the organisation stops describing its experience surfaces and starts *knowing* them. That transition — from episodic observation to cumulative knowledge — is the real value Decipher delivers.

What This Is Not

Decipher is not a CX platform. It does not replace existing analytics, VoC programmes, or survey infrastructure. Those systems continue to generate the signals that feed it.

Decipher is not an AI product. LLMs are used at the ingestion layer and within the PDCA execution cycle. The architecture's value is structural, not inferential.

Decipher is not a dashboard or a reporting tool. It produces no commentary, no narrative summaries, no insight decks.

Decipher is the missing computational layer between human experience signals and organisational action: the substrate that makes experience drivers persistent, stateful, actionable, measurable, and cumulative.

It is infrastructure. And like all foundational infrastructure, its value becomes invisible once it exists — because without it, nothing above it works properly.

The Category of Innovation

The most precise analogy for what Decipher is doing is Edgar Codd's relational model.

In 1970, data existed in application-specific formats. Every system had custom query logic. There was no standard structure, no portability, no shared truth. Codd's contribution was not a better database. It was the formal definition of a primitive, a set of invariants, and a query model that made data computation possible across applications and organisations.

The result was not a product. It was infrastructure that the entire software industry reorganised around.

Decipher is making the equivalent move for experience drivers. The primitive is defined. The invariants are enforced. The query model exists. The domain: human experience as it occurs across any system where humans encounter organisations is now computationally addressable in a way it has never been before.

CX practitioners have organised around drivers for twenty years. They were right. What they lacked was the substrate.

Decipher redeems the driver. It does not replace the intuition - it operationalises it.

Current State

The architecture is conceptually validated and computationally implemented at prototype stage across:

- Vertical-specific ingestion and mention extraction
- Deterministic SEU computation engine
- OU clustering and generation logic
- Problem statement generation with full source lineage
- PDCA cycle execution framework with enterprise knowledge base integration architecture
- Closed-loop elasticity computation
- Institutional memory schema
- End-to-end simulation across healthcare and financial services verticals

Production API, enterprise SDK, and multi-tenant deployment are the next phase.

The architecture is not a proposal. It is a working model awaiting production engineering and the first enterprise pilot that will generate the empirical proof the foundation deserves.

The Question This Work Raises

The simplest way to understand Decipher is through a single question:

If organisations already know that drivers are the real determinants of human experience — why are those drivers still treated as disposable analytical artifacts rather than governed operational entities?

Everything in this architecture is an answer to that question.

Two years of work. One structural gap. One missing layer.

— Raheem Asghar

For technical architecture detail, schema specification, vertical implementation examples, and execution layer documentation — available on request.